CHAPTER 2: SYSTEM ANALYSIS & DESIGN

PHASE I: SYSTEM ANALYSIS

I. INTRODUCTION

System analysis and design of a blockchain-based self-sovereign digital identity security management system (DISMS) involves a comprehensive approach to developing a secure, decentralized, and user-centric solution for managing digital identities. This process integrates principles from blockchain technology, cryptographic protocols, and identity standards such as Decentralized Identifiers (DIDs) and Verifiable Credentials (VCs) to enable individuals to securely control and share their identity data. It translates the 'what' defined by stakeholders and research into the 'how' of building the actual system.

II. METHODOLOGY

The system analysis for this project employed a hybrid methodology, combining elements of **Structured Analysis** with principles of **User-Centered Design**.

- **Structured Analysis:** Techniques such as process modeling (implicitly through use case analysis) and data modeling (defining requirements for VCs and DIDs) were used to systematically understand and document the system's functions and data requirements. This provides a clear and organized view of the system's components and interactions.

- **User-Centered Design Principles:** Given the focus on user control and empowerment inherent in SSI, a user-centered approach was integrated. This involved focusing on the needs, perspectives, and potential challenges faced by the primary users (Cameroonian citizens) through techniques like questionnaire surveys. The goal was to ensure the requirements reflect real-world usability and address user concerns regarding privacy and control.

The overall process followed these steps:

1. **Requirement Analysis:** Employing techniques like literature review, analysis of existing CNI processes, and questionnaire surveys to gather information.

2. **Analysis:** Analyzing the gathered information to identify specific functional requirements, non-functional requirements, constraints, and problems with the existing system.
3. **Design:** Architecting how the system will be built
4. **Implementation:** Writing code based on the design and building the application
5. **Test:** Checking the system for errors and validating it meets requirements.
6. **Maintain**: Ongoing support and updates after deployment.
7. **Commission / Verification**: Final testing, acceptance, and system delivery.

This combined approach aimed to ensure both technical rigor and a strong focus on the end-user experience and needs.

1. System Development Lifecycle (The Hybrid Model)

The system will follow **the Hybrid methodology**, encompassing primarily Agile and Waterfall, to leverage the strengths of each.

- **Waterfall Methodology** is a traditional approach that follows a linear sequence of phases: planning, design, development, testing, and deployment. Each phase ends before the next one starts thus providing a clear roadmap.
- **Agile Methodology** focuses on iterative development and continuous improvement. Projects are broken into smaller chunks called “sprints” with frequent delivery and feedback loops to enable quick adaptation based on learning.

Nonetheless, within each phase, practices such as sprints or even daily stand-up meetings that are agile can be incorporated for the iterative development of deliverables within the larger project framework.

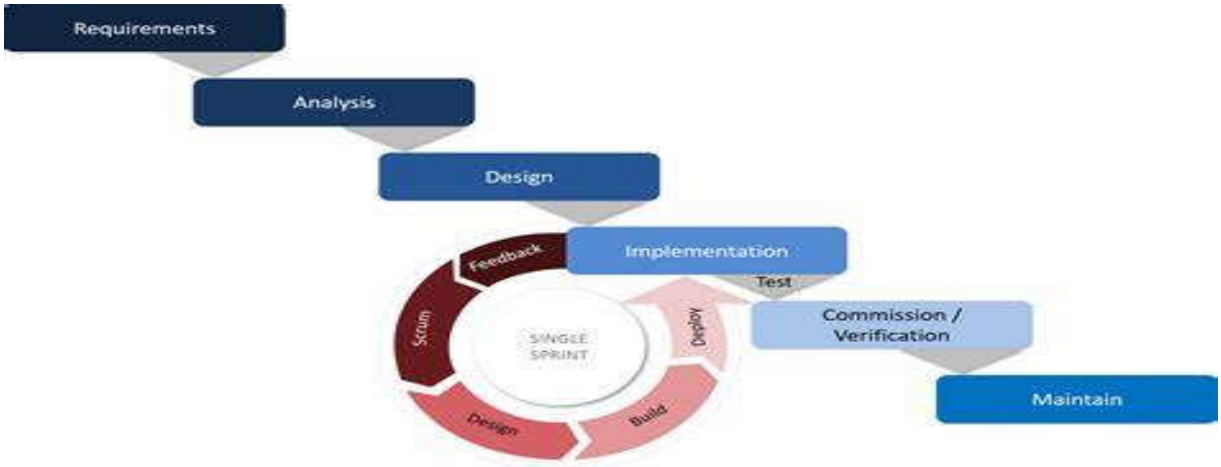
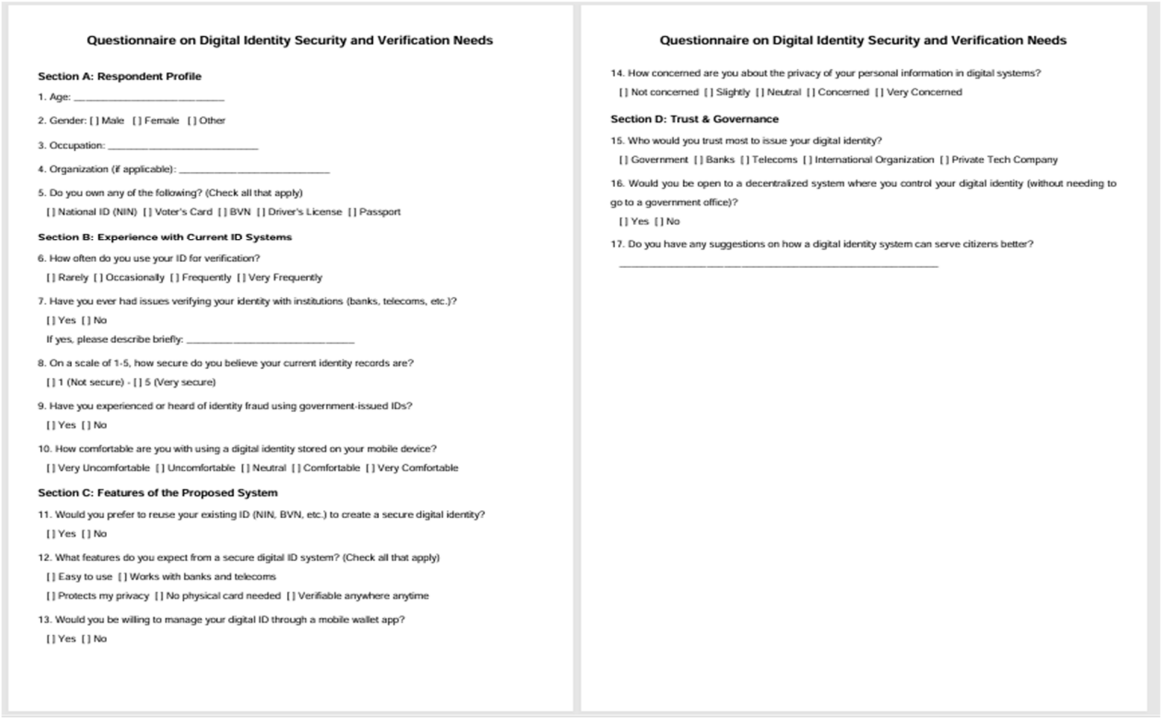


Figure 10: Hybrid Model

2. Requirement Gathering Techniques

- **Literature Review:** An extensive review of academic papers, technical standards industry reports, news articles, and case studies (e.g., Estonia's e-ID system) was conducted.

- **Questionnaire Survey:** A questionnaire was designed to collect data from users of the system



Questionnaire on Digital Identity Security and Verification Needs

Section A: Respondent Profile

1. Age: _____
2. Gender: [ ] Male [ ] Female [ ] Other
3. Occupation: _____
4. Organization (if applicable): _____
5. Do you own any of the following? (Check all that apply)
[ ] National ID (NIN) [ ] Voter's Card [ ] BVN [ ] Driver's License [ ] Passport

Section B: Experience with Current ID Systems

6. How often do you use your ID for verification?
[ ] Rarely [ ] Occasionally [ ] Frequently [ ] Very Frequently
7. Have you ever had issues verifying your identity with institutions (banks, telecoms, etc.)?
[ ] Yes [ ] No
If yes, please describe briefly: _____
8. On a scale of 1-5, how secure do you believe your current identity records are?
[ ] 1 (Not secure) - [ ] 5 (Very secure)
9. Have you experienced or heard of identity fraud using government-issued IDs?
[ ] Yes [ ] No
10. How comfortable are you with using a digital identity stored on your mobile device?
[ ] Very Uncomfortable [ ] Uncomfortable [ ] Neutral [ ] Comfortable [ ] Very Comfortable

Section C: Features of the Proposed System

11. Would you prefer to reuse your existing ID (NIN, BVN, etc.) to create a secure digital identity?
[ ] Yes [ ] No
12. What features do you expect from a secure digital ID system? (Check all that apply)
[ ] Easy to use [ ] Works with banks and telecoms
[ ] Protects my privacy [ ] No physical card needed [ ] Verifiable anywhere anytime
13. Would you be willing to manage your digital ID through a mobile wallet app?
[ ] Yes [ ] No

Questionnaire on Digital Identity Security and Verification Needs

14. How concerned are you about the privacy of your personal information in digital systems?
[ ] Not concerned [ ] Slightly [ ] Neutral [ ] Concerned [ ] Very Concerned

Section D: Trust & Governance

15. Who would you trust most to issue your digital identity?
[ ] Government [ ] Banks [ ] Telecoms [ ] International Organization [ ] Private Tech Company
16. Would you be open to a decentralized system where you control your digital identity (without needing to go to a government office)?
[ ] Yes [ ] No
17. Do you have any suggestions on how a digital identity system can serve citizens better?
_____

Figure 11: Questionnaire Sample

- **Analysis of Existing System(s):** The current, largely manual process of using and verifying the Cameroonian National Identity card (CNI) was analyzed based on information gathered from the

literature review, general knowledge of administrative processes, and insights potentially derived from the questionnaire responses regarding user experiences.

III. SYSTEM REQUIREMENTS

1. Functional Requirements

Functional requirements define the specific actions, tasks, or functions the system must be capable of performing. They are based on the interactions between the actors (Citizen/Holder, Issuer, Verifier) and the system.

Functional requirement	Detail explanation
1. User Registration and Identity Onboarding	• Authenticate users using existing national identifiers. • Retrieve and verify identity details from respective government databases. • Generate a unique **Decentralized Identifier (DID)** for each user. • The system must facilitate the setup of the citizen's digital wallet application.
2. Verifiable Credential (VC) Issuance	• Issue cryptographically signed **Verifiable Credentials** after validation. • Bind credentials to user's DID and wallet. • The system must enable Issuers to securely transmit the issued VC to the corresponding citizen's digital walle
3. Digital Wallet	• Provide users with a digital wallet to store and manage DIDs and VCs. • Enable secure credential sharing with verifiers. • The wallet must provide mechanisms for backup and recovery of DIDs/keys/VCs under the user's control.

Table 3: Functional Requirements

2. Non-Functional Requirements

Non-functional requirements define the quality attributes, performance characteristics, and constraints of the system. They describe how the system should perform its functions

Functional requirement	Detail explanation

3. Security	• Use end-to-end encryption for data at rest and in transit. • Apply digital signatures to credentials. • Prevent unauthorized access using MFA and biometric options.
4. Privacy & Data Protection	• Comply with NDPR (Nigeria), GDPR (EU), and other relevant regulations. • Support **Zero-Knowledge Proofs (ZKPs)** for privacy-preserving verification. • Ensure that no personal data is stored on-chain (only hashes or proofs).
5. Scalability	• Support high concurrency of user requests. • Scale to millions of users and credentials with minimal performance degradation.
6. Performance	• Ensure identity verification and VC issuance complete in under 3 seconds. • Optimize blockchain transactions for speed and cost-efficiency.
7. Availability & Reliability	• The blockchain network must be highly available, ensuring that DID resolution and VC revocation status checks can be performed reliably. • The system components (wallet app, issuer/verifier services) should be robust and handle errors gracefully.
8. Maintainability	• The system code (smart contracts, backend, frontend) must be well-documented, modular, and follow good coding practices to facilitate future maintenance and updates.

Table 4: Non-Functional Requirements

3. Actors and Their Use Cases.

- **Primary Actors**: These are the direct users of the system who initiate or are most affected by system actions. They are: **User**, **Credential issuer**, **Verifier**
- **Secondary Actors:** These actors support the system, provide infrastructure, or play oversight roles. They are: **System Administrator**, **Blockchain Network**.

Actors	Use Cases
1. User/Holder	• Register and onboard using existing ID (e.g., NIN, BVN). • Generate a Decentralized Identifier (DID) linked to verified personal data. • Receive Verifiable Credentials (VCs) issued by trusted authority. • Store Credentials securely in a digital wallet. • Revoke or Update Credentials when needed
1. Credential Issuer	• Authenticate User Identity using existing ID database. • Issue VCs (e.g., Proof of NIN, Voter ID, BVN). • Sign Credentials Cryptographically and register hashes on blockchain. • Revoke Credentials when invalidated (e.g., fraud, expired). • Update Credential Information upon request (e.g., address change).
2. Verifier	• Request Identity Verification from user. • Verify the Authenticity of submitted credentials via blockchain. • Check Credential Revocation Status. • Grant Access to Services if identity is verified (e.g., open bank account, issue SIM card).
3. System Administrator	• Configure and Maintain Infrastructure (blockchain nodes, APIs, wallet services). • Manage Credential Issuer Access and permissions. • Audit Platform Usage and activity logs. • Ensure Security Compliance and regulatory reporting. • Deploy Smart Contracts and perform upgrades or bug fixes.
4. Blockchain Network (Smart Contract)	• Log Credential Issuance and Revocation securely and immutably. • Enforce Access Control and privacy rules via smart contracts. • Serve as Decentralized Trust Layer for all identity interactions. • Handle VC Revocation Registry

Table 5: Actors and Use Cases

4. User Stories of the System.

❖ **Identity Holder (User):**

- As an Identity Holder, I want to register my digital identity using my official documents, so that I can control who accesses my data.
- As an Identity Holder, I want to view who accessed my data and when, so I can track usage.
- As an Identity Holder, I want to grant or revoke access to my identity data, so I can maintain privacy.
- As an Identity Holder, I want to view my credentials and their status, so I can verify their validity.

❖ **Issuer:**

- As an Issuer, I want to verify user-provided information, so I can confirm their identity.
- As an Issuer, I want to issue a digitally signed credential, so the identity holder can prove authenticity.
- As an Issuer, I want to revoke credentials if they are outdated or fraudulent, so I can maintain trust.

❖ **Verifier:**

- As a Verifier, I want to request verification of credentials from users, so I can confirm their legitimacy.
- As a Verifier, I want to receive validated, tamper-proof identity information, so I can trust the verification process.
- As a Verifier, I want to confirm access rights via smart contracts, so I only access authorized data.

❖ **System Administrator:**

- As an Administrator, I want to manage role assignments (Issuer, Verifier), so that only authorized entities can access system features.
- As an Administrator, I want to monitor network activity, so I can detect anomalies.
- As an Administrator, I want to deploy and upgrade smart contracts, so the system remains functional and secure.

Use case diagram illustration

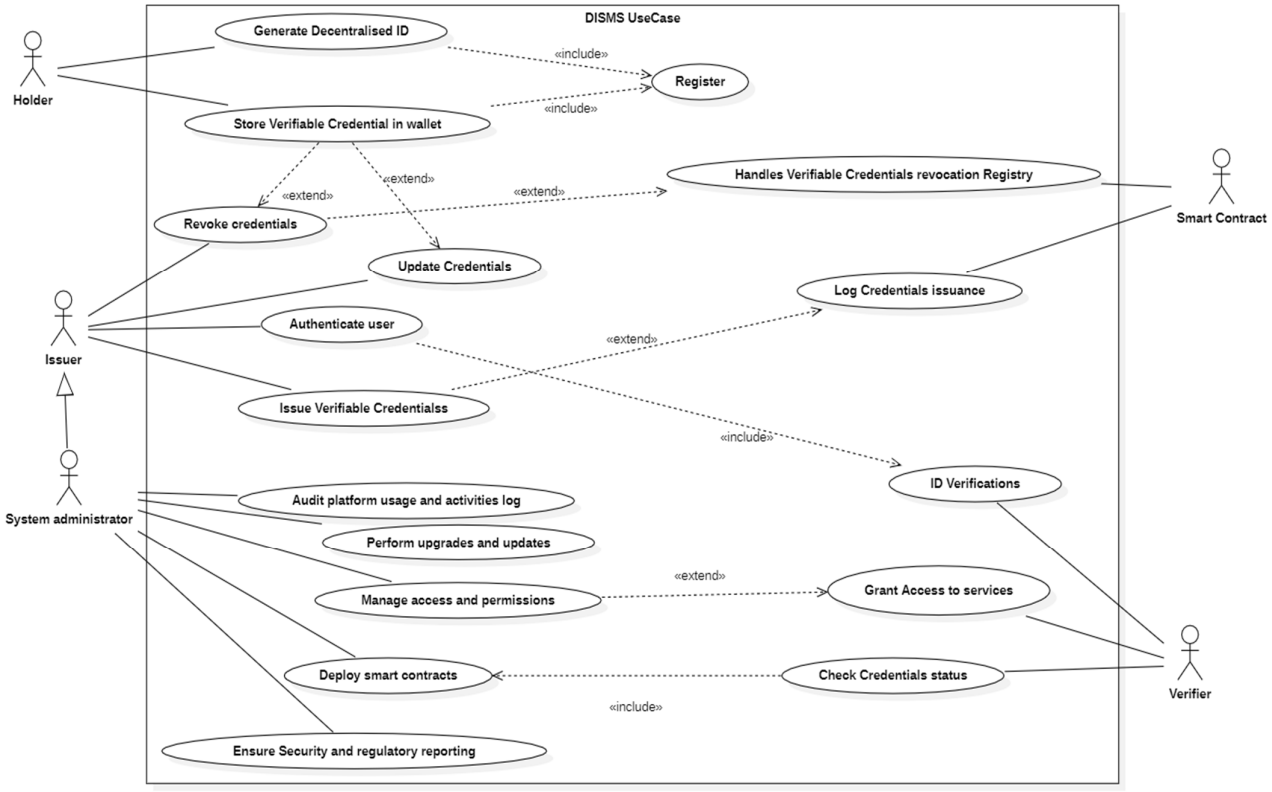


Figure 12: UseCase Diagram

PHASE II: SYSTEM DESIGN & ARCHITECTURE

IV. SYSTEM ARCHITECTURE

Defines the high-level structure of the Digital Identity Security Management System (DISMS), outlining its major components, their relationships, and how they interact to fulfill the system's requirements. A well-designed architecture is crucial for ensuring the system is scalable, maintainable, secure, and effectively implements the principles of Self-Sovereign Identity (SSI) while integrating with existing Cameroonian identity verification processes. A layered architectural approach is proposed for clarity and modularity.

1. Architectural Overview (Layered Approach)

The proposed architecture can be visualized in logical layers:

1. **Presentation Layer:** This layer encompasses the user interfaces through which actors interact with the system.
    - ***Citizen/Holder/User****:* Digital Wallet Application (likely mobile, potentially web) providing functionalities for managing DIDs, VCs, keys, consent, and presenting credentials.
    - ***Issuer****:* Secure Web Portal or Application Interface for authorized personnel to manage the VC issuance process (including triggering existing ID verification).
    - ***Verifier****:* Secure Web Portal or API Interface for organizations to request and receive verifiable presentations, and initiate the verification process.
    - ***System Administrator****:* Management Console for overseeing the blockchain network and participant permissions.
2. **aplication Layer:** This layer contains the backend business logic, APIs, and SDKs that orchestrate the system's functions and mediate communication between the presentation layer and the blockchain layer.
    - *Wallet Backend Services (Optional):* May handle synchronization or communication aspects for the wallet.
    - *Issuer Service:* Contains logic for handling VC requests, interfacing with the Existing ID Verification Module, constructing VCs, signing them, and interacting with the blockchain (via SDK) to issue them or record necessary data.
    - *Verifier Service:* Contains logic for defining verification policies, requesting presentations, receiving them, interacting with the blockchain (via SDK) to resolve DIDs and check VC status (revocation), and performing cryptographic verification.
    - *Blockchain SDK/API:* Provides standardized methods for application components to interact with the blockchain network (submit transactions, query ledger state).
3. **Blockchain Layer:** This is the core decentralized trust layer of the system.
    - *Blockchain Network Nodes:* Peers/Validators operated by authorized participants (Issuers, Verifiers, potentially governance bodies) that maintain copies of the ledger and participate in consensus.
    - *Smart Contracts (Chaincode):* Deployed on the network, containing the immutable logic for DID registration/resolution, VC revocation registry management, VC schema registry (optional), and potentially access control.

- *Ledger:* The distributed, immutable ledger storing transaction records, DID-related data (anchors, public keys, service endpoints via DID Documents or pointers), and VC revocation statuses. **Crucially, sensitive Personal Identifiable Information (PII) is NOT stored on the ledger.**

4. **Data Storage:** Data is stored both on-chain and off-chain.
   - *On-Chain:* Immutable ledger data (transaction history, DID registry data, VC revocation status).
   - *Off-Chain (User Wallet):* User's private keys, full VCs, personal settings. This data is controlled by the user.
   - *Off-Chain (Issuer/Verifier Systems):* Application-specific data, logs, potentially cached data (managed securely according to data protection regulations). Existing government identity databases remain off-chain, accessed via secure interfaces.

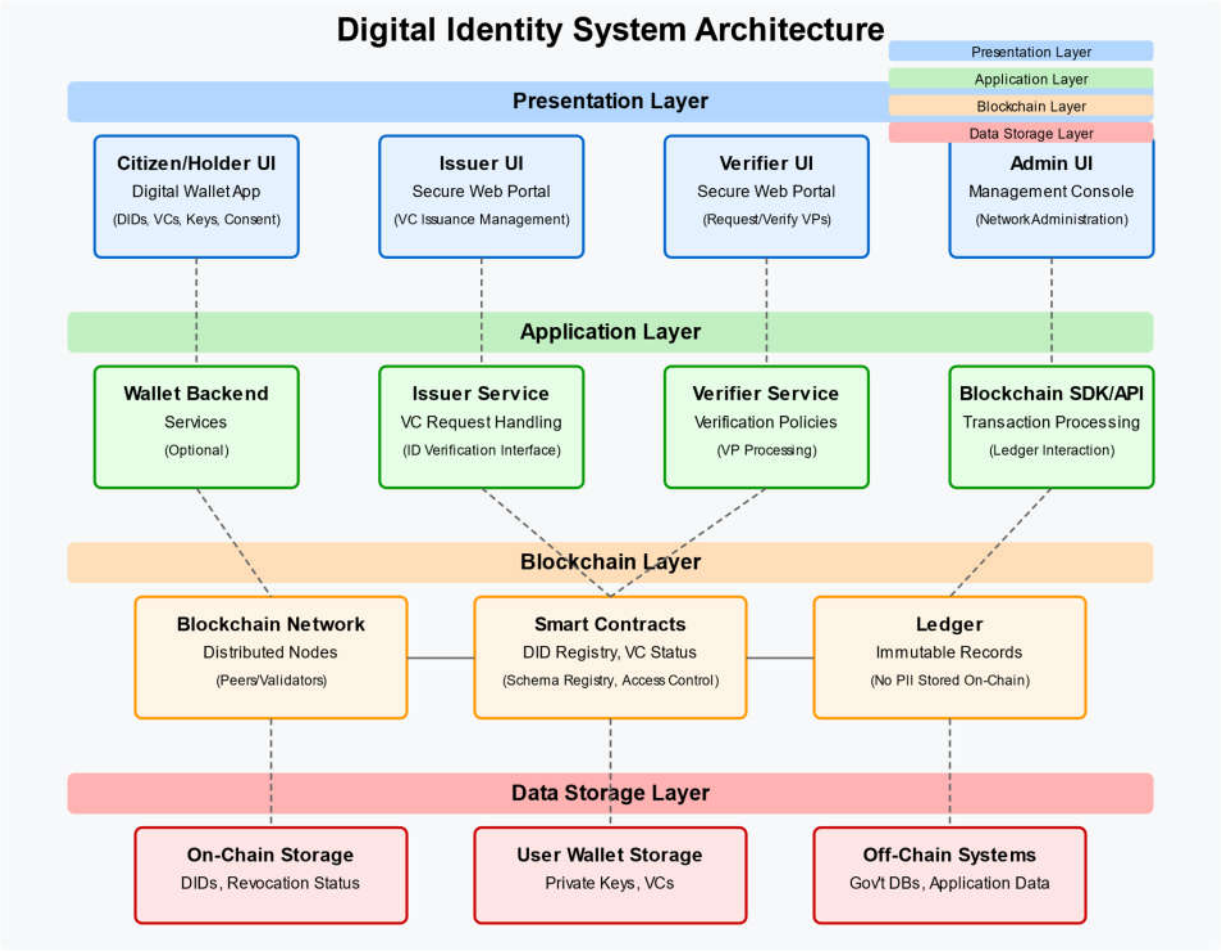


Figure 13: Architectural diagram

2. Key Components and Interaction.

- **Digital Wallet:** The primary user agent. Communicates securely (potentially using protocols like DIDComm) with Issuer and Verifier services and interacts with the blockchain via the SDK/API (often indirectly through backend services or directly for certain operations). Manages user keys and consent

- **Issuer Service:** Interacts with the **Existing ID Verification Module** (a critical component representing the secure interface to official government databases or verification processes for the CNI, etc.). Upon successful off-chain verification, it uses the Blockchain SDK to issue VCs or anchor relevant data.
- **Verifier Service:** Receives presentations from the Wallet, uses the Blockchain SDK to fetch necessary public keys (via DID resolution) and check VC revocation status from the blockchain ledger, performs cryptographic checks, and returns the verification result.
- **Blockchain Network:** Provides the decentralized trust infrastructure. Smart contracts execute predefined logic automatically. Nodes ensure consensus and maintain the ledger's integrity.
- **Existing ID Verification Module:** This conceptual component represents the secure gateway or process through which the authorized Issuer confirms the validity of a citizen's existing ID against the authoritative source. This interaction is primarily off-chain but is a prerequisite for issuing trusted VCs within this system.

2. Blockchain Design

This section elaborates on the specific design choices and characteristics of the blockchain layer implemented for the Digital Identity Security Management System (DISMS). The blockchain serves as the decentralized trust anchor for managing Decentralized Identifiers (DIDs) and the status of Verifiable Credentials (VCs).

2.1. Block Structure:

The fundamental unit of the blockchain ledger is the block, which groups together validated transactions. Based on the common structure illustrated in the image each block in this conceptual model consists of two main parts: a **Header** and a **Body**.

Block Header: Contains metadata essential for linking blocks and ensuring the integrity of the chain. The key fields shown in the reference image are:

1. **Previous Block Address (Previous Block Hash):** This field stores the cryptographic hash of the header of the immediately preceding block in the chain. This hash acts as a pointer, creating the sequential link between blocks (Block n-1 -> Block n -> Block n+1). Tampering with any previous block would change its hash, invalidating this pointer and all subsequent blocks, thus ensuring chain integrity.
2. **Timestamp:** Records the approximate time the block was created or validated by the network's consensus mechanism. This provides a chronological order to the blocks and transactions.
3. **Nonce:** Stands for "Number used once." This field is primarily associated with Proof-of-Work (PoW) consensus mechanisms, where miners repeatedly change the nonce until the block header's hash meets a specific difficulty target. Even in systems not using PoW (like the likely choice for this DISMS), a nonce field might be included for other cryptographic purposes or compatibility reasons inherited from the base platform design.
4. **Merkel Root (Merkle Root):** This is the root hash of a Merkle tree constructed from all the individual transactions included in the block's body. The Merkle tree allows for efficient verification that a specific transaction is part of the block without needing to process the entire list of transactions. It ensures the integrity of the transaction set within the block.

Block Body (Implied): Although not detailed in the reference image structure, the block body contains the actual list of validated transactions that have been grouped into this block. In the context of the DISMS, these transactions would represent actions like **DID registrations** (**registerDID**), **DID updates** (**updateDID**), VC revocation status changes (**revokeVC**), etc.

This structure, particularly the linking via the Previous Block Address (hash) and the integrity checks provided by the Merkel Root, forms the basis of the blockchain's security and immutability.

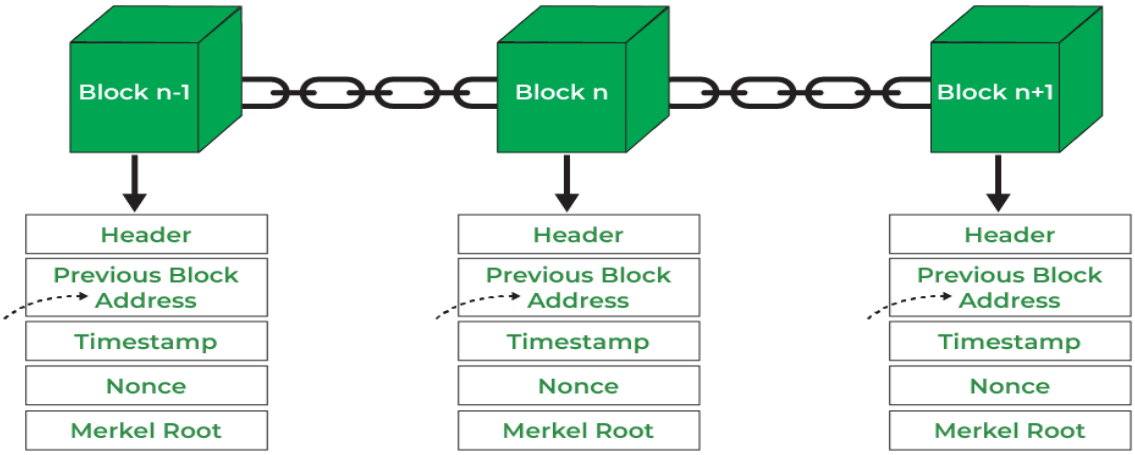


Figure 14: Blocks in Blockchain

- Chain Structure: The blockchain is a linear sequence of blocks, where each block is cryptographically linked to the previous one through its hash. This creates an immutable and tamper-evident history of all identity-related transactions. Modifying any block would require regenerating all subsequent blocks, which is computationally infeasible on a live chain.

2.2. Consensus Mechanism:

- This determines how new blocks are created and validated by the participating nodes in the network, ensuring agreement on the state of the ledger. Given the requirement for a permissioned network involving trusted entities, a consensus mechanism suitable for such environments is necessary..

  - **Proof-of-Authority (PoA):** This is a highly suitable mechanism for permissioned blockchains. In PoA, only pre-approved, authorized nodes (e.g., operated by government agencies or trusted institutions) have the right to create new blocks and validate transactions. This offers high transaction throughput and is energy-efficient. It aligns well with the need for known, accountable participants in an identity system.

  - *Less Likely for this specific use case:* **Proof-of-Work (PoW)** (used by Bitcoin) or **Proof-of-Stake (PoS)** (used by Ethereum 2.0) are typically used in public, open networks to prevent Sybil attacks and achieve consensus among unknown participants. While robust, their resource intensity (PoW) or complexity (PoS) and open nature might be less ideal for a system requiring controlled access and high transaction speed for identity lookups and verifications. For your report, briefly

mentioning them and explaining why PoA (or a similar consortium-based mechanism like Delegated Proof-of-Stake - DPoS) is more appropriate demonstrates a deeper understanding.

Traditional **Proof-of-Work** (PoW) is generally unsuitable due to its energy consumption and permissionless nature.

2.3. Genesis Block Creation:

The genesis block is the very first block in the blockchain. It is statically created and hardcoded into the blockchain software. For this identity system, the genesis block would:

- Initialize the state of the ledger (e.g., register the initial authorized government authority nodes).
- Contain the initial configuration parameters for the network and smart contracts.
- Its hash serves as the root for the entire chain.

2.4. Block Validation Rules:

Nodes in the network follow specific rules to validate incoming blocks before adding them to their copy of the chain. These rules include:

- Verifying the cryptographic integrity of the block header and the link to the previous block.
- Validating all transactions included in the block according to the system's rules and smart contract logic (e.g., checking if the transaction is signed by an authorized party, if the identity hash format is correct).
- Ensuring the consensus mechanism's requirements are met (e.g., the block was created by an authorized validator in PoA).

As outlined in the Literature Review (Section III.4), a **Permissioned Blockchain** platform (e.g., Hyperledger Fabric, a private/consortium Ethereum network) is recommended for this DISMS.

- **Justification:**
  - ***Control & Governance****:* Allows restricting participation to authorized entities (government Issuers, registered Verifiers), aligning with the need for trust and accountability in national identity systems.
  - ***Privacy****:* Enables better control over data visibility compared to public blockchains. Transaction details can often be confined to relevant parties.

- ***Performance**:* Generally offers higher transaction throughput and lower latency than public blockchains, suitable for potentially high volumes of identity transactions.
- ***Regulatory Compliance**:* Easier to align with data protection regulations when participation and data access are controlled.

3. Architectural diagrams

Component Diagram: Shows the major software components (Wallet App, Issuer Service, Verifier Service, Smart Contracts, Blockchain Nodes, Existing ID Verification Module) and their dependencies/interfaces.

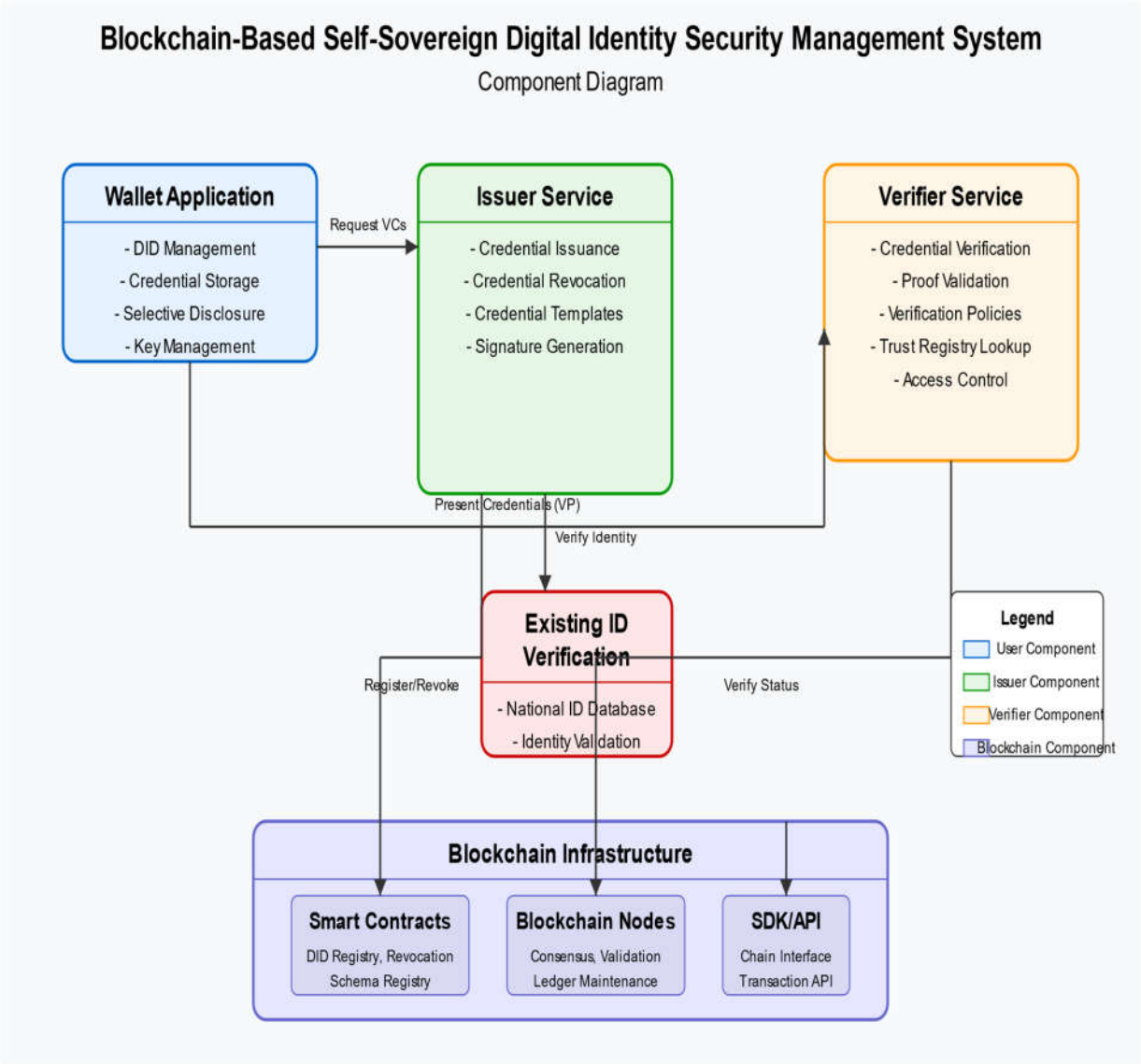


Figure 15: Component Diagram

Sequence Diagrams: Detail the interactions between components for key use cases like onboarding, VC issuance, and verification.

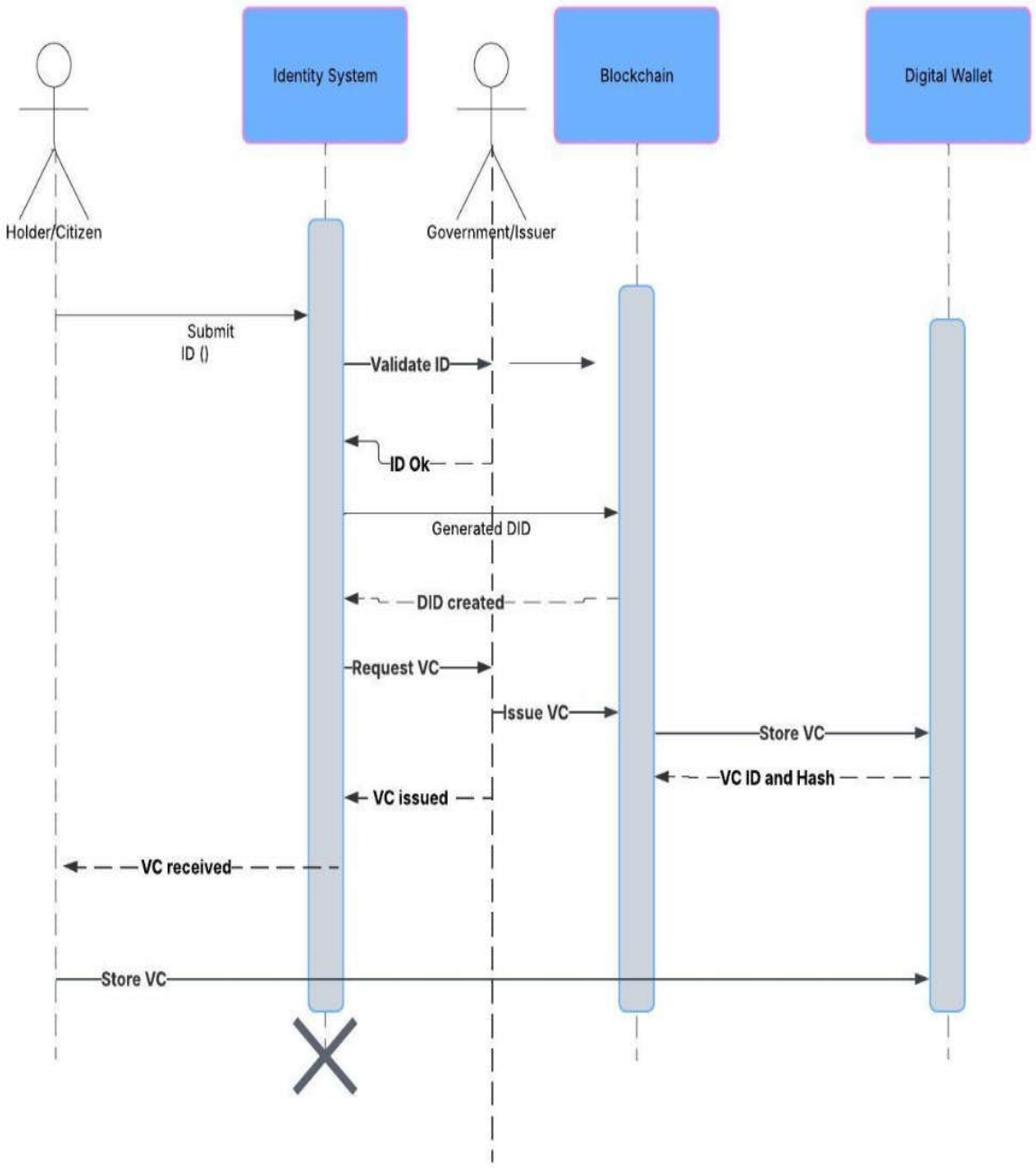


Figure 16: Holder Sequence Diagram

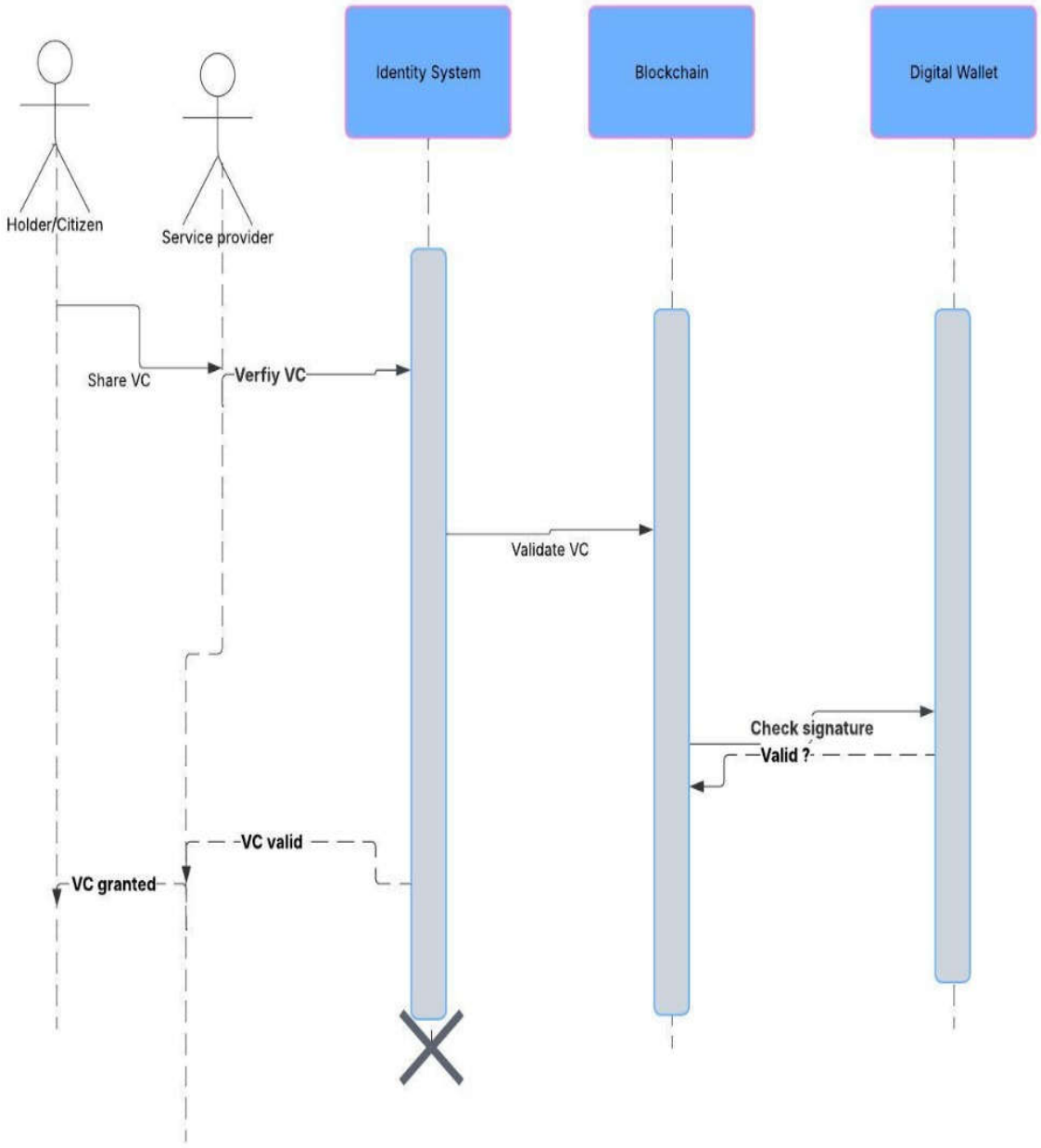


Figure 17: Service Sequence diagram

Class Diagrams: Describes the attributes and operations of a class and also the constraints imposed on the system

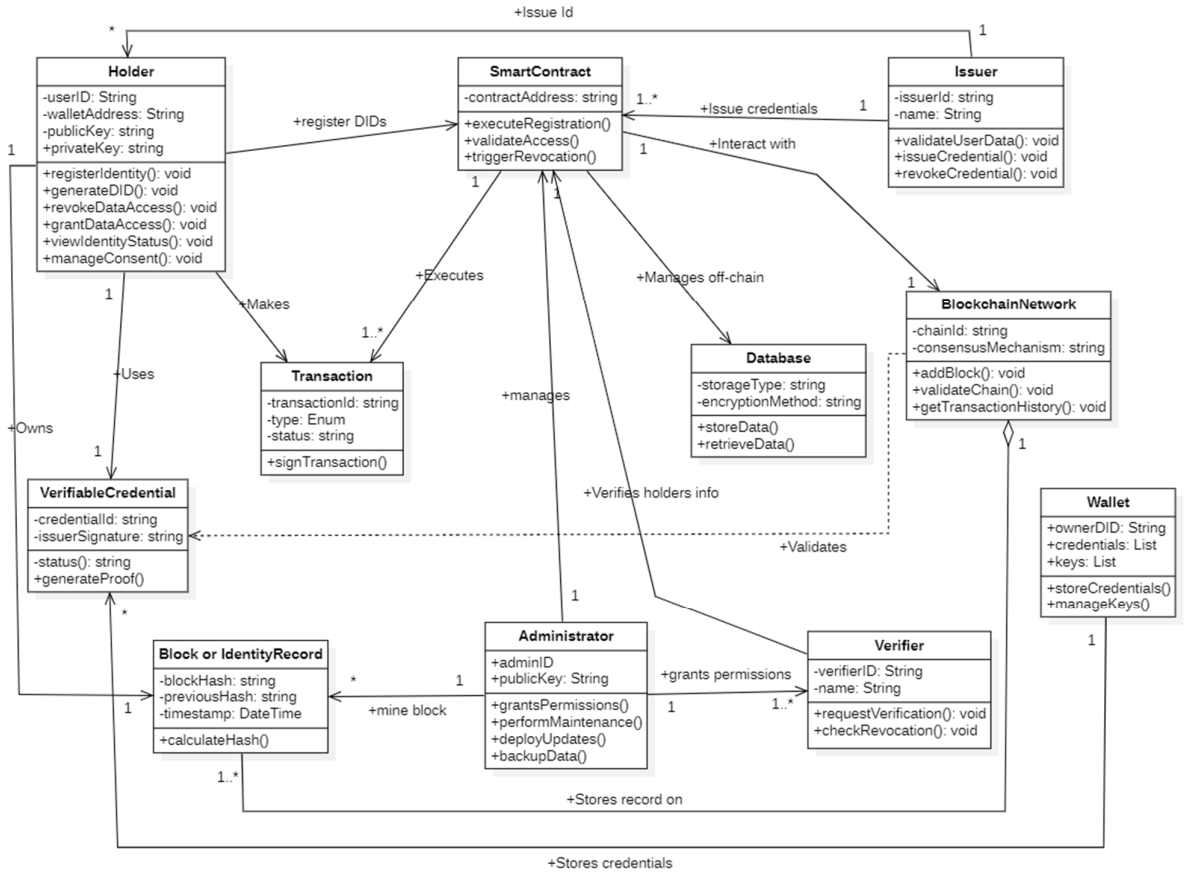


Figure 18: Class Diagram

This layered architecture provides a modular and robust foundation for the DISMS, separating concerns and facilitating the integration of user interfaces, application logic, blockchain technology, and essential links to existing identity verification processes in Cameroon.

This deployment diagram illustrates the physical architecture of the blockchain-based self-sovereign digital identity management system for Cameroon. It shows how different components are distributed across physical infrastructure and how they communicate.

V. SECURITY DESIGN

Given that the Digital Identity Security Management System (DISMS) handles sensitive identity information and relies on cryptographic guarantees, a comprehensive and multi-layered security design is

paramount. Security must be integrated throughout the architecture, encompassing the underlying blockchain network, smart contracts, application services, communication channels, and user-facing components like the digital wallet. This section outlines the key security considerations and measures designed into the system.

1. Cryptographic Foundations

Cryptography is the bedrock upon which the security and trust of the DISMS are built. Standard, well-vetted cryptographic algorithms and protocols are employed:

- **Hashing:** Secure hash algorithms (e.g., SHA-256) are used for ensuring data integrity, creating cryptographic commitments, linking blocks in the blockchain, and potentially representing sensitive identifiers (like the National ID number) in VCs without storing the raw value.

- **Digital Signatures:** Asymmetric cryptography (e.g., Ed25519 or ECDSA) is used extensively for:
  - *VC Issuance:* Issuers sign VCs with their private key, allowing Verifiers to confirm authenticity using the Issuer's public key (retrieved via their DID).
  - *VC Presentation:* Holders sign Verifiable Presentations with their private key, proving control over the presented VCs and the DID.
  - *Blockchain Transactions:* Transactions submitted to the blockchain network (e.g., DID registrations, VC status updates) are signed by the originating entity to ensure authenticity and non-repudiation.

- **Encryption:**
  - *Data in Transit:* Transport Layer Security (TLS/SSL) is used to encrypt all communication between system components (Wallet-Issuer, Wallet-Verifier, Application-Blockchain Node) over networks, preventing eavesdropping and tampering.
  - *Data at Rest (Wallet):* Sensitive data stored within the user's digital wallet (especially private keys) must be encrypted using strong symmetric encryption algorithms (e.g., AES-256), with the decryption key derived from user authentication factors (PIN, biometrics).

2. Access Control Mechanisms

Access control ensures that only authorized entities can perform specific actions or access certain resources:

- **Blockchain Network Level (Permissioned):**
  - ***Participant Onboarding****:* Strict procedures (managed by the System Administrator/Governing Body) for onboarding trusted organizations (Issuers, Verifiers) onto the network, typically involving identity verification and issuance of digital certificates (e.g., via MSP in Hyperledger Fabric).
  - ***Smart Contract Permissions****:* Smart contracts enforce rules based on the transaction submitter's identity (e.g., only an authorized Issuer DID can call the issueVC function for a specific VC type; only the DID owner can update their DID Document). Access Control Lists (ACLs) or role-based logic within smart contracts restrict function invocation.
  - ***Channel Permissions (Fabric):*** Data privacy can be enhanced using channels, restricting transaction visibility to only the authorized participants of that channel.
- **Application Level:**
  - ***Issuer/Verifier Portals****:* Strong authentication mechanisms for personnel accessing Issuer and Verifier applications/portals. Role-based access control within these applications limits user capabilities based on their responsibilities.
- **Wallet Level:**
  - ***User Authentication****:* Secure mechanisms (PIN, password, biometrics) protect access to the wallet application itself and authorize sensitive operations like key usage (signing presentations).

3. Privacy-Preserving Techniques

Protecting user privacy is a core requirement, mandated by regulations (like Law No. 2010/012) and central to the SSI philosophy:

- **Data Minimization:** VCs are designed to contain only necessary claims. The system supports selective disclosure, allowing users (via their wallets) to share only the specific claims required by a Verifier, rather than the entire credential.
- **Off-Chain Data Storage:** Sensitive Personal Identifiable Information (PII) derived from existing IDs is *not* stored on the blockchain ledger. It resides primarily within the user's wallet (under their

control) or temporarily within secure Issuer/Verifier systems during processing, subject to data protection policies. Hashes or cryptographic commitments may be stored on-chain if needed for verification logic.

- **DID Correlation Resistance:** Users should be able to use different DIDs for different contexts or relationships (pairwise DIDs) to prevent tracking and correlation of their activities across various services. The wallet should facilitate the management of multiple DIDs.
- **Transaction Privacy (Blockchain):** Depending on the chosen permissioned blockchain platform, mechanisms like private data collections (Fabric) or private transactions can further limit the visibility of transaction details even among network participants.

4. Secure Key Management

Since control in SSI is tied to cryptographic keys, their management is critical:

- **Key Generation:** Keys must be generated using cryptographically secure random number generators directly within the secure environment of the user's wallet.
- **Key Storage:** Private keys must be stored with the highest level of security possible on the user's device, ideally utilizing hardware-backed secure elements (e.g., Secure Enclave on iOS, Trusted Execution Environment on Android, TPM on PCs, or hardware wallets). Software-only storage must use strong encryption tied to user authentication.
- **Backup and Recovery:** User-controlled mechanisms for wallet backup (e.g., encrypted backup files) and key recovery are essential. Users must be educated about their responsibility in securely managing these backups. Issuer/system providers should *not* have access to user private keys.

5. Smart Contract Security

Smart contracts execute autonomously on the blockchain and handle critical logic; thus, their security is vital:

- **Auditing:** Smart contracts must undergo thorough security audits by independent experts before deployment to identify vulnerabilities (e.g., re-entrancy, integer overflow/underflow, denial-of-service vectors, incorrect access control logic).
- **Secure Coding Practices:** Adherence to established secure development guidelines for the specific smart contract language (e.g., Solidity, Go chaincode) is mandatory. This includes input validation, proper state management, and checks-effects-interactions patterns.

- **Testing:** Rigorous unit testing, integration testing, and potentially formal verification should be performed.
- **Upgradability:** Implementing secure patterns for smart contract upgradability (e.g., proxy patterns) allows for fixing bugs or adding features post-deployment without disrupting the entire system, but the upgrade process itself must be securely governed.

VI. DEPLOYMENT DESIGN

This section describes how the different software and blockchain components of your system would be deployed and interact in a production environment.

PWA Deployment Strategy

A Progressive Web App is essentially a website built with technologies that provide an enhanced user experience, including offline capabilities, installability, and push notifications. The deployment strategy will focus on making the frontend accessible via the web while ensuring the backend and blockchain components are robust and scalable.

- **Frontend (PWA):**
    - **Hosting:** The static assets of your PWA (HTML, CSS, JavaScript, images, service worker) will be hosted on a standard web server or a Content Delivery Network (CDN). CDNs are particularly useful for distributing content globally, ensuring faster loading times for users regardless of their location. Services like Netlify, Vercel, AWS S3 with CloudFront, or Firebase Hosting are excellent choices for PWA hosting.
    - **Service Worker Deployment:** The service worker, a crucial part of a PWA enabling offline functionality and other enhancements, will be registered and served as part of your PWA's static assets. It will be responsible for caching assets, handling network requests, and potentially managing push notifications.
    - **HTTPS:** Serving the PWA over HTTPS is mandatory as service workers require a secure origin. This ensures the integrity and confidentiality of data transmitted between the user and the server.

- **Backend Application Server:**
    - **Hosting:** The backend server, responsible for handling API requests from the PWA, business logic, and interaction with the blockchain, will need a reliable hosting environment. Options include:
        - **Cloud Platforms:** AWS (EC2, Elastic Beanstalk, Lambda), Google Cloud Platform (Compute Engine, App Engine, Cloud Functions), Microsoft Azure (Virtual Machines, App Service, Azure Functions). These platforms offer scalability, reliability, and various managed services.
        - **Dedicated Servers or VPS:** For more control over the infrastructure, you could opt for dedicated servers or Virtual Private Servers (VPS). However, this requires more manual configuration and management.
        - **Containerization (Docker):** Packaging your backend application in a Docker container allows for consistent deployment across different environments. Container orchestration platforms like Kubernetes or Docker Swarm can manage scaling and deployment of containerized applications.
- **Blockchain Engine (Custom):**
    - **Node Deployment:** Your custom blockchain engine will need to run on a network of nodes. These nodes will be responsible for maintaining the distributed ledger, processing transactions, and achieving consensus.
    - **Server Infrastructure:** Each blockchain node will require a server with sufficient computational resources, storage, and network bandwidth. These servers can be:
        - **Cloud-based Virtual Machines:** Similar to the backend server, cloud platforms offer scalable and reliable infrastructure for blockchain nodes.
        - **Dedicated Hardware:** For higher security and performance, especially for consensus participants, dedicated hardware might be considered.

- **Network Configuration:** Proper network configuration is crucial for the peer-to-peer communication between blockchain nodes. This includes setting up firewall rules and ensuring nodes can discover and connect to each other.

- **Consensus Mechanism Considerations:** The deployment might vary slightly depending on your chosen consensus mechanism (PoW or PoA). PoW requires more computational power for mining nodes, while PoA necessitates a well-defined and managed set of authority nodes.

- **Off-Chain Database (if used):**

  - **Managed Database Services:** Cloud providers offer managed database services (e.g., AWS RDS, Google Cloud SQL, Azure Database) that handle backups, scaling, and maintenance. Choose a database type (SQL or NoSQL) that suits your data storage needs.

  - **Self-Managed Databases:** You can also set up and manage your own database servers on cloud VMs or dedicated hardware.

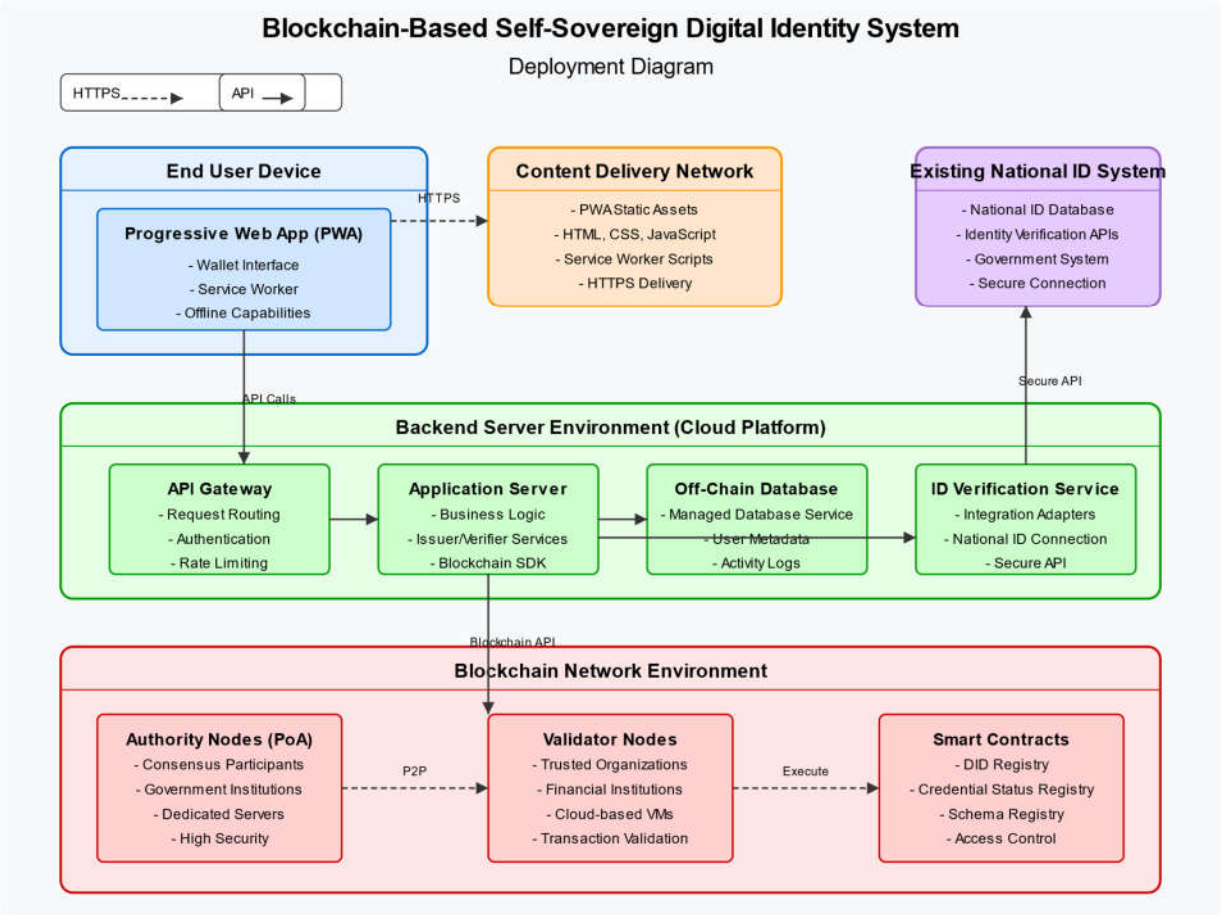
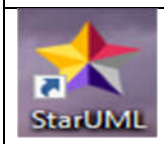


Figure 19: Deployment Diagram

TOOLS AND TECHNOLOGIES USED

- **UML / Diagramming:** Example: StarUML / lucid chart*/* was used for creating UML diagrams (Use Case, Sequence, Component, Class).

Table 6: Tools and Technologies

TOOLS	EXPLANATION
	A Gantt chart is a project management tool that helps in planning, scheduling and monitoring a project
	**StarUML** is an open-source project designed to provide a fast, flexible, extensible, and feature-rich UML/MDA platform
	Used to analyze large volumes of user feedback, surveys, or government documents, functional and non-functional requirements.